

Cuestionario Técnico de Control N.º 1

Structs, Punteros y Aplicación al Proyecto Integrador Anual

Grupo: Orvion Robotics

Integrantes: Malek Cheheid, Teo Laguna y Santiago Osma

Secundaria Técnica 1 de Vicente Lopez

Laboratorio de Programación

Yamil Ganduglia, Mansilla Muñoz York Elías

Repositorio: <https://github.com/teolaguna/SISTEMAPROYECTO.git>

Fecha de Entrega: 25 de Septiembre de 2026

Cuestionario Técnico de Control N.º 1

1. Heterogeneidad: ¿Qué ventaja ofrece agrupar los datos de un sensor o dispositivo en un struct en lugar de declarar muchas variables sueltas?

Un struct permite reunir bajo un mismo nombre datos de distinto tipo (enteros, cadenas, decimales) que en conjunto describen a una misma entidad real, como un sensor o un dispositivo. Esto evita la dispersión de variables independientes que, aunque pertenezcan al mismo elemento físico, quedarían sueltas y sin relación explícita en el código. Al agruparlas, se logra mayor orden, legibilidad y mantenibilidad: se puede pasar la entidad completa como un único parámetro a una función, copiarla, almacenarla en arreglos o enviarla por punteros, en lugar de manejar cada variable por separado. Además, reduce errores, ya que si se agrega un nuevo dato del sensor, simplemente se suma un miembro a la estructura sin tener que modificar cada función que la utiliza.

2. Operador Flecha: ¿Por qué el compilador de C++ no permite usar la notación de punto (ptr.id) cuando la variable es de tipo puntero?

La notación de punto (.) se utiliza para acceder a los miembros de una variable que representa directamente un objeto o estructura. Un puntero, en cambio, no es la estructura en sí, sino que almacena una dirección de memoria donde esa estructura está alojada. Por lo tanto, escribir ptr.id sería un error porque se estaría intentando acceder a un miembro “id” del puntero mismo (que no existe, ya que el puntero solo contiene una dirección), y no de la estructura a la que apunta. El operador flecha (->) existe justamente para resolver esto en un solo paso: primero desreferencia el puntero (accede a la dirección de memoria que contiene) y luego accede al miembro correspondiente. Es equivalente a escribir (*ptr).id, pero de forma más clara y directa.

3. Eficiencia en RAM: ¿Por qué el pasaje por referencia mediante punteros optimiza la memoria procesada frente al pasaje por copia?

Cuando una estructura se pasa por copia a una función, el programa reserva un nuevo espacio en memoria y duplica allí todos los datos de la estructura original, lo cual consume más memoria RAM y tiempo de procesamiento, sobre todo si la estructura es grande o contiene muchos miembros. En cambio, al pasar un puntero, lo único que se copia es la dirección de memoria (un valor pequeño, de tamaño fijo, independientemente de lo grande que sea la estructura). La función trabaja directamente sobre el dato original, sin duplicarlo. Esto no solo ahorra memoria, sino que también permite que los cambios realizados dentro de la función se reflejen en la variable original, ya que ambas hacen referencia al mismo espacio físico de memoria.

4. Buffer de Entrada: ¿Qué comportamiento erróneo ocurre en la terminal si se omite la instrucción `cin.ignore()` antes de un `getline()`?

Cuando se utiliza `cin >>` para leer un dato numérico, el usuario presiona Enter para confirmarlo, pero ese carácter de salto de línea (`\n`) queda pendiente en el buffer de entrada, ya que `cin >>` no lo consume. Si a continuación se llama a `getline()` sin limpiar antes el buffer con `cin.ignore()`, la función `getline()` va a leer inmediatamente ese salto de línea sobrante en lugar de esperar a que el usuario escriba el texto. El resultado visible en la terminal es que el programa “se salta” la lectura del nombre o texto esperado, mostrando el campo vacío o pasando directamente a la siguiente instrucción sin darle al usuario la oportunidad real de ingresar el dato.

5. Aplicación al Proyecto Integrador: ¿Qué entidad o componente hardware de su Proyecto Integrador Anual representaron con esta estructura y qué variables componen sus miembros?

En nuestro Proyecto Integrador Anual —un robot con microcontrolador (Arduino), sensor ultrasónico, servomotor para una tapa de descarga y motores DC controlados por un driver— identificamos una oportunidad para aplicar esta lógica: actualmente el código

maneja el estado del sistema de detección mediante variables globales sueltas (distancia, estaDetectando, yaAbrioTapa, tiempoDeteccion). Proponemos agruparlas en un struct llamado EstadoSensor, donde distancia representaría la lectura del sensor ultrasónico en centímetros, estaDetectando indicaría si actualmente hay un obstáculo delante, yaAbrioTapa si la tapa ya se abrió para ese evento de detección, y tiempoDeteccion el instante en milisegundos en que se detectó el obstáculo, usado para calcular el tiempo de espera antes de abrir la tapa. Con este cambio, las funciones dejarían de modificar variables globales dispersas y en su lugar recibirían un puntero a esta estructura (EstadoSensor* ptr), lo que permitiría leer y actualizar el estado real del sistema sin duplicarlo en memoria, concentrando en un único lugar toda la información del ciclo de detección, apertura de tapa y control de motores.

Aporte en el trabajo

Integrante	Aporte en el trabajo
Malek Cheheid	Elaboración del Informe y Manual de programador
Teo Laguna	Repositorio y Ejecución del código
Santiago Osma	Redacción teórica y cuestionario